Lakshya

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Internal Contract Document

Definition of Done for June 30th

Two people open the app on different networks. Both log in. One creates an instant meeting and sees the share link panel. They copy the link and send it. The other person opens the link, enters the waiting room, gets admitted. Both see each other's video and hear each other's audio. Both can mute, toggle cameras, send chat messages, write agenda, send reactions. The host can end the meeting. Both get redirected to the dashboard. Meeting appears in history.

That is the complete working prototype.

Remarks:

1) This is the first remark, Radhe Radhe.
2)

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Patharr ki lakeer

1) No one writes code they cannot explain. If you cannot explain why a line exists, you do not write it.

2) This document is the single source of truth. No API endpoint, socket event name, hook interface, or database table changes without updating this document first and informing the team.

3) No one waits silently when blocked by any task. If you are stuck for more than 30 minutes, you message the team immediately.

4) Every Friday all three of us run the full system together and test the integrated flow.

5) AI is allowed only for understanding concepts and resolving doubts. Zero AI generated code. Every line is written and understood by the person who wrote it.

Phase 1

Team

Name	Role	Owns
Sweety	Backend Part	Auth, User, Meeting REST APIs, Database, Supabase, Upstash
Savita	Frontend Part	All pages, components, UI, consuming hooks and APIs
Sahil	WebRTC Pipeline	Signaling server, WebRTC hooks, integration, deployment

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Frontend_ICD

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

What I Will Deliver to Savitaaa

1. Three Hooks She Must Use Exactly As Documented

These are the only three files that connect the UI to WebRTC and Socket.IO. Savita does not modify these files. If you need something changed, you will tell me first.

Hook 1) useMediaDevices()

Manages camera and microphone access.

How to use:

```
import useMediaDevices from '../hooks/useMediaDevices';

const {
  localStream,
  isMuted,
  isCameraOff,
  toggleMute,
  toggleCamera,
  error
} = useMediaDevices();
```

What each thing is:

`localStream` — a MediaStream object from the user's camera and microphone. Attach this to a video element to show the user their own preview. Will be null until the user grants camera permission.

`isMuted` — boolean. True means the microphone is currently off. Use this to show the correct mute button state.

`isCameraOff` — boolean. True means the camera is currently off. Use this to show a blank tile or camera off icon.

`toggleMute` — function. Call this when the user clicks the mute button. No arguments needed.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

`toggleCamera` — function. Call this when the user clicks the camera button. No arguments needed.

`error` — string or null. If camera or microphone access was denied this will contain an error message to display to the user.

How to attach localStream to a video element:

```
import { useEffect, useRef } from 'react';

const localVideoRef = useRef(null);

useEffect(() => {
  if (localVideoRef.current && localStream) {
    localVideoRef.current.srcObject = localStream;
  }
}, [localStream]);

// In JSX
<video
  ref={localVideoRef}
  autoPlay
  muted
  playsInline
/>
```

The `muted` attribute on the local video is mandatory. Without it the user hears their own voice echoed back. `autoPlay` is needed so the video plays without the user clicking. `playsInline` is needed for iOS Safari.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Hook 2) useSocket(token)

Manages the Socket.IO connection to the backend.

How?

```
import useSocket from '../hooks/useSocket';
import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

const { token } = useContext(AuthContext);

const {
  socket,
  isConnected,
  emit
} = useSocket(token);
```

What each thing is:

`socket` — the raw Socket.IO socket object. Savita uses this to listen for events with `socket.on('event:name', handler)`. She must remove listeners in useEffect cleanup with `socket.off('event:name', handler)`.

`isConnected` — boolean. True when the socket connection to the backend is active.

`emit` — function. Shorthand for `socket.emit`. Takes event name and payload. Use this to send events.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

How to listen for events:

```
useEffect(() => {
  if (!socket) return;

  const handleUserJoined = (data) => {
    console.log(data.userName, 'joined');
  };

  socket.on('room:user-joined', handleUserJoined);

  return () => {
    socket.off('room:user-joined', handleUserJoined);
  };
}, [socket]);
```

Savitaa suno Always clean up listeners in the return function of useEffect. If you do not, the same event fires multiple times because old listeners stack up.

Hook 3) useWebRTC(localStream, roomCode, socket)

Manages all peer to peer video connections.

How to use:

```
import useWebRTC from '../hooks/useWebRTC';

const { remoteStreams, connectionStates } = useWebRTC(
  localStream,
  roomCode,
  socket
);
```

Pass `localStream` from `useMediaDevices`, `roomCode` from the URL params, and `socket` from `useSocket`.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

What each thing is:

`remoteStreams` — an array of objects. Each object represents one remote participant currently in the room. Map over this array to render video tiles.

Each object in the array looks like this:

```
{
  userId: "uuid string",
  userName: "string",
  stream: MediaStream
}
```

`connectionStates` — an object where keys are userIds and values are connection state strings, `connecting`, `connected`, `disconnected`. Use this to show a loading indicator on a video tile while connection is being established.

How to render remote streams:

```
{remoteStreams.map((remote) => (
  <VideoTile
    key={remote.userId}
    stream={remote.stream}
    userName={remote.userName}
    connectionState={connectionStates[remote.userId]}
  />
))}
```

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Inside VideoTile, how to attach a remote stream:

```
const VideoTile = ({ stream, userName, connectionState }) => {
  const videoRef = useRef(null);

  useEffect(() => {
    if (videoRef.current && stream) {
      videoRef.current.srcObject = stream;
    }
  }, [stream]);

  return (
    <div>
      <video ref={videoRef} autoPlay playsInline />
      <span>{userName}</span>
    </div>
  );
};
```

Dhyan dene wali baat:- no `muted` attribute on remote video tiles. You want to hear other people.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

2. Socket Events (Important)

These are the events she listens for and emits. She does not invent new event names. She uses exactly these strings. All event name constants are in `src/utils/constants.js` — she imports from there, never types the string directly.

Events to emit (things you will send to the server):

Event	When	Payload
room:join	User enters meeting room	`{ roomCode, userId, userName }`
room:leave	User clicks leave	`{ roomCode, userId }`
media:mute-toggle	User clicks mute	`{ userId, isMuted }`
media:camera-toggle	User clicks camera	`{ userId, isCameraOff }`
chat:send-message	User sends chat message	`{ roomCode, content }`
chat:typing	User is typing	`{ roomCode, userId }`
agenda:save	User saves agenda	`{ roomCode, content }`
reaction:send	User sends reaction	`{ roomCode, emoji }`
host:mute-all	Host mutes everyone	`{ roomCode }`
host:remove-participant	Host removes someone	`{ roomCode, targetUserId }`
host:end-meeting	Host ends meeting	`{ roomCode }`

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Events you will listen for:

Event	When it arrives	Payload
room:participants	Right after joining	`{ participants: [] }`
room:user-joined	Someone joins	`{ userId, userName, isHost }`
room:user-left	Someone leaves	`{ userId, userName }`
media:state-changed	Someone mutes or toggles cam	`{ userId, isMuted, isCameraOff }`
chat:new-message	New chat message arrives	`{ senderId, senderName, content, sentAt }`
chat:user-typing	Someone is typing	`{ userId, userName }`
agenda:updated	Agenda was saved	`{ content, updatedBy, updatedAt }`
reaction:received	Someone reacted	`{ userId, userName, emoji }`
meeting:ended	Host ended meeting	no payload
waiting-room:admitted	Host admitted you	no payload
waiting-room:rejected	Host rejected you	no payload
error:unauthorized	You tried a host action	`{ message }`

(THE ABOVE TWO IMG'S ARE AI GENERATED AND HENCE MAY HAVE ERROR)

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

API Endpoints

These come from the backend. You will call them through the service files I set up.never writes fetch calls directly in components, Savitaaa always fetch calls through the service layer.

Auth endpoints:

`POST /api/auth/register` — body: `{ name, email, password }` — returns token and user

`POST /api/auth/login` — body: `{ email, password }` — returns token and user

`GET /api/auth/me` — header: Authorization Bearer token — returns current user

User endpoints:

`GET /api/users/profile` — returns full profile

`PATCH /api/users/profile` — body: `{ name, avatarUrl }` — returns updated profile

Meeting endpoints:

`POST /api/meetings/instant` — no body — returns meeting with roomCode and join link

`POST /api/meetings/create` — body: `{ title }` — returns meeting with roomCode and join link

`GET /api/meetings/my` — returns array of past meetings

`GET /api/meetings/:roomCode/validate` — returns whether room is valid and active

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

4. What You Must Never Do

1) never call `socket.emit` with an event name typed as a raw string. Always import from constants.

2) never attach a stream to a video element without checking that both the ref and the stream exist first.

3) never forget the `muted` attribute on the local video element.

4) never forget to clean up socket event listeners in useEffect return functions.

5) never store the JWT token in localStorage. It lives in AuthContext in memory only.

6) never make API calls directly inside a component. Always use the service layer.

Backend_ICD

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

What I Will Deliver to Sweety

1. The Running Server

I will give Sweety a working `server.js` that starts Express, attaches Socket.IO, and connects to the database and Redis. She does not set up the server. She only writes modules that plug into it.

What she will receive on Thursday(I will keep updating my code):

backend/

1) server.js I write this (IWT)

2)package.json (IWT)

3)env.example (IWT)

4)src/

4.1) config/

4.1.1) env.js (IWT)

4.1.2) db.js (IWT)

4.1.3) redis.js (IWT)

4.2) middleware/

4.2.1) auth.js (IWT)

4.2.2) errorHandler.js (IWT)

She received this as a GitHub repo. She clones it, fills in the `.env` file, runs `npm install` and `npm run dev`, and the server starts. Her job is then to build her modules inside `src/modules/`.

2. The auth.js Middleware (We will use JWT here (now Savitaa and Sweety figure out why?)

This is the JWT verification middleware Sweety will apply to every protected route. She does not write it. She imports it.

How?:

```
const auth = require('../../middleware/auth');

router.get('/profile', auth, userController.getProfile);
router.post('/meetings/create', auth, meetingController.create);
```

What it does:
Reads the Authorization header, extracts the Bearer token, verifies it against the JWT secret, and attaches the decoded user to `req.user`. If the token is missing or invalid it sends 401 automatically and her controller never runs.

What `req.user` contains after auth middleware runs:

```
req.user = {
  userId: "uuid string",
  email: "string",
  name: "string"
}
```

She can use `req.user.userId` in any controller to know which user made the request. She never reads `req.headers.authorization` herself.

3. The apiResponse Utility

Every response Sweety sends must use these two functions. She does not write her own response format.

How?

```
4529  const { successResponse, errorResponse } = require('../../utils/apiResponse');
4530
4531  // In a controller
4532  successResponse(res, { user: userData }, 201);
4533  errorResponse(res, 'EMAIL_ALREADY_EXISTS', 'An account with this email already exists', 409);
```

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Success response shape this produces:

```
{
  "success": true,
  "data": { }
}
```

Error response shape this produces:

```
{
  "success": false,
  "error": "ERROR_CODE_IN_CAPS",
  "message": "Human readable message"
}
```

Sweety you will never write `res.status(200).json(...)` manually. Always uses these two functions.

4. The Standard Error Codes Sweety Must Use

Use Standard Error codes, I am sure you are aware of codes like 201, 200 OK, 500, 401 etc etc. (If now just ping me on grp I will send one list)

5. The Route Registration Pattern

I already registered all her routes in `server.js`. you do require to touch `server.js`. only tells me the path prefix for your router and I will add one line. (yes use it for the learning purpose)

My side in server.js:

javascript

```
app.use('/api/auth', require('./src/modules/auth/auth.routes'));
app.use('/api/users', require('./src/modules/user/user.routes'));
app.use('/api/meetings', require('./src/modules/meeting/meeting.routes'));
```

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

Her(Sweety) side in each routes file, you exports Express router:

```
const router = require('express').Router();
// her routes here
module.exports = router;
```

6. What I Expect Sweety to Deliver to Me

I need these things from Sweety to make the signaling server work correctly.

From auth module - a working `POST /api/auth/login` and `POST /api/auth/register` that return a valid JWT token in this exact shape:

```
{
  "success": true,
  "data": {
    "token": "jwt string here",
    "user": {
      "userId": "uuid",
      "name": "string",
      "email": "string"
    }
  }
}
```

I will use this token to authenticate Socket.IO connections. If the token shape is different, socket auth breaks.

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

From meeting module — a working `POST /api/meetings/instant` and `POST /api/meetings/create` that return a meeting object in this exact shape:

```
{
  "success": true,
  "data": {
    "meetingId": "uuid",
    "roomCode": "abc-defg",
    "title": "string",
    "hostId": "uuid",
    "status": "active"
  }
}
```

I will use `roomCode` as the Socket.IO room identifier. If this field is missing or named differently, the entire room system breaks.

From the meeting module — a working `GET /api/meetings/:roomCode/validate` endpoint that returns whether a room code is valid and the meeting is active. I call this before admitting someone to a Socket.IO room.

```
{
  "success": true,
  "data": {
    "valid": true,
    "meetingId": "uuid",
    "hostId": "uuid",
    "title": "string"
  }
}
```

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

DATABASE

We will use Supabase here (search why? I will ask these things in order to make sure you are reading this document (it takes a lot of time to write and organise)).

Run these in Supabase SQL editor in this exact order (as there are dependent on each other)

001) users table

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  avatar_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

002) meetings table

```
CREATE TABLE meetings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(255) NOT NULL,
  room_code VARCHAR(20) UNIQUE NOT NULL,
  host_id UUID REFERENCES users(id) ON DELETE CASCADE,
  status VARCHAR(20) DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ
);
```

Last edit:- 17 June (1530hrs By SAHIL CHAUDHARY)

003) agendas table

```
CREATE TABLE agendas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
  content TEXT,
  last_updated_by UUID REFERENCES users(id),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

004) messages table

```
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
  sender_id UUID REFERENCES users(id),
  content TEXT NOT NULL,
  sent_at TIMESTAMPTZ DEFAULT NOW()
);
```

Savita's Corner

Sweety's Corner

designing_Decisions

Under progress……..